BRACT's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Practical No: 5

Division: C Batch: 2

Roll No: 3090367 PRN No: 12415134

Name of Student: Surabhi Harish Patel

Subject Name: Deep Learning

Problem Statement:

Implement and compare RNN, LSTM, and GRU models for sequence classification, and analyze their performance using appropriate evaluation metrics.

Algorithm :

Dataset Description
The **Reuters Newswire dataset** contains news articles categorized into **46 different topics**. Each news article is represented as a sequence of integer word IDs. The objective is to predict the category/topic of the news article.
For example, categories may represent topics related to:

- Economics
- Trade
- Politics
- Business
- Commodities
- Finance
- etc.

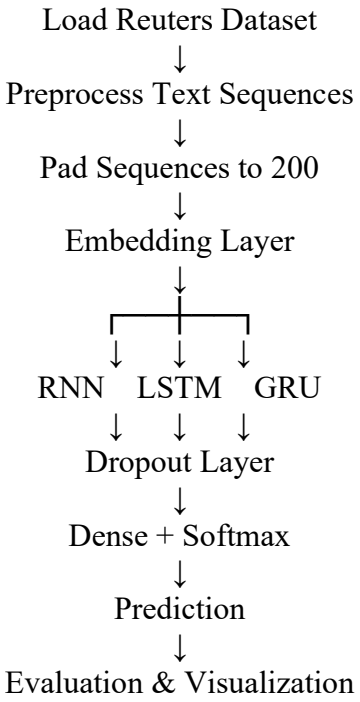
The dataset is suitable for comparing recurrent neural network architectures because the input is sequential text data.

RNN
RNNs are designed for sequential data. They maintain a hidden state that carries information from previous time steps. However, standard RNNs can suffer from the vanishing-gradient problem, making it difficult to learn long-term dependencies.

LSTM
LSTM is an improved RNN architecture that uses memory cells and gates to retain important information for longer periods. It contains forget, input, and output gates.

GRU
GRU is a simpler gated recurrent architecture. It uses update and reset gates to control information flow and can handle long-term dependencies with fewer parameters than LSTM in many configurations.

Methodology:



Evaluation Metrics

- **Accuracy:** Percentage of correctly classified samples.
- **Precision:** Correct positive predictions among all positive predictions.
- **Recall:** Correctly identified samples among all actual samples.
- **F1-score:** Balance between precision and recall.
- **Confusion Matrix:** Shows correct and incorrect predictions for each class.
- **Training Time:** Measures computational efficiency.

Results:

Model	Accuracy	Precision	Recall	F1
RNN	50.76%	41.20%	50.76%	42.23%
LSTM	55.08%	50.68%	55.08%	**50.96%**
GRU	**56.14%**	48.75%	**56.14%**	50.72%

Code :

```
import kagglehub
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import os

from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import SimpleRNN, LSTM, GRU, Dense

path = kagglehub.dataset_download(
    "rizkykiky/gold-price-dataset"
)

print("Path to dataset files:", path)

print("Files in dataset folder:")
print(os.listdir(path))

df = pd.read_csv(
    os.path.join(path, "Daily.csv")
)

print(df.head())
print(df.columns)
print(df.shape)

df["Date"] = pd.to_datetime(df["Date"])
```

```
df = df[["Date", "INR"]]

df = df.dropna()

df = df.sort_values("Date")

# Remove commas and convert INR to numeric
df["INR"] = df["INR"].str.replace(",", "", regex=False)

df["INR"] = pd.to_numeric(
    df["INR"],
    errors="coerce"
)

df = df.dropna(subset=["INR"])

print(df.head())
print(df.shape)

plt.figure(figsize=(12, 5))

plt.plot(
    df["Date"],
    df["INR"]
)

plt.xlabel("Date")
plt.ylabel("Gold Price (INR)")
plt.title("Gold Price Trend")

plt.grid(True)
plt.show()

prices = df["INR"].values.reshape(-1, 1)

scaler = MinMaxScaler()

prices_scaled = scaler.fit_transform(prices)

print("Original price range:",
      prices.min(), "to", prices.max())

print("Scaled price range:",
      prices_scaled.min(), "to", prices_scaled.max())
```

```
sequence_length = 30

X = []
y = []

for i in range(
    sequence_length,
    len(prices_scaled)
):
    X.append(
        prices_scaled[
            i-sequence_length:i
        ]
    )

    y.append(
        prices_scaled[i]
    )

X = np.array(X)
y = np.array(y)

print("X shape:", X.shape)
print("y shape:", y.shape)

train_size = int(len(X) * 0.8)

X_train = X[:train_size]
X_test = X[train_size:]

y_train = y[:train_size]
y_test = y[train_size:]

print("Training data:", X_train.shape)
print("Testing data:", X_test.shape)

def build(kind):

    model = Sequential()

    if kind == "RNN":

        model.add(
            SimpleRNN(
                50,
```

```
                input_shape=(
                    X_train.shape[1],
                    X_train.shape[2]
                )
            )
        )

    elif kind == "LSTM":

        model.add(
            LSTM(
                50,
                input_shape=(
                    X_train.shape[1],
                    X_train.shape[2]
                )
            )
        )

    else:

        model.add(
            GRU(
                50,
                input_shape=(
                    X_train.shape[1],
                    X_train.shape[2]
                )
            )
        )

    model.add(Dense(1))

    model.compile(
        optimizer="adam",
        loss="mean_squared_error"
    )

    return model

results = []
predictions_dict = {}

for kind in ["RNN", "LSTM", "GRU"]:
```

```
    print("\n")
    print(kind)


    model = build(kind)

    history = model.fit(
        X_train,
        y_train,
        epochs=10,
        batch_size=32,
        validation_split=0.2,
        verbose=1
    )

    predictions = model.predict(
        X_test,
        verbose=0
    )

    predicted_prices = scaler.inverse_transform(
        predictions
    )

    actual_prices = scaler.inverse_transform(
        y_test
    )

    predictions_dict[kind] = predicted_prices

    mae = mean_absolute_error(
        actual_prices,
        predicted_prices
    )

    rmse = np.sqrt(
        mean_squared_error(
            actual_prices,
            predicted_prices
        )
    )

    r2 = r2_score(
        actual_prices,
        predicted_prices
```

```
    )

    print("MAE:", mae)
    print("RMSE:", rmse)
    print("R2:", r2)

    results.append([
        kind,
        mae,
        rmse,
        r2
    ])

plt.figure(figsize=(14, 6))

plt.plot(
    test_dates,
    actual_prices.flatten(),
    label="Actual Price"
)

plt.plot(
    test_dates,
    predictions_dict["RNN"].flatten(),
    label="RNN Prediction"
)

plt.plot(
    test_dates,
    predictions_dict["LSTM"].flatten(),
    label="LSTM Prediction"
)

plt.plot(
    test_dates,
    predictions_dict["GRU"].flatten(),
    label="GRU Prediction"
)

plt.xlabel("Date")
plt.ylabel("Gold Price (INR)")
plt.title("Actual vs Predicted Gold Prices")

plt.legend()
plt.grid(True)
```

```
plt.show()

results_df = pd.DataFrame(
    results,
    columns=[
        "Model",
        "MAE",
        "RMSE",
        "R2"
    ]
)

print(results_df)

plt.figure(figsize=(15, 5))

# MAE
plt.subplot(1, 3, 1)

plt.bar(
    results_df["Model"],
    results_df["MAE"]
)

plt.title("MAE Comparison")
plt.xlabel("Model")
plt.ylabel("MAE")
plt.grid(axis="y")

# RMSE
plt.subplot(1, 3, 2)

plt.bar(
    results_df["Model"],
    results_df["RMSE"]
)

plt.title("RMSE Comparison")
plt.xlabel("Model")
plt.ylabel("RMSE")
plt.grid(axis="y")

# R2
plt.subplot(1, 3, 3)
```

```
plt.bar(
    results_df["Model"],
    results_df["R2"]
)

plt.title("R² Comparison")
plt.xlabel("Model")
plt.ylabel("R² Score")
plt.grid(axis="y")

plt.suptitle(
    "Performance Comparison of RNN, LSTM and GRU",
    fontsize=16
)

plt.tight_layout()
plt.show()

"""* Lower MAE and RMSE indicate better performance, while higher R² indicates
better performance.
* The model with the best overall combination of these metrics is considered the
best-performing model.




"""
```

Output:
Loading Reuters dataset...

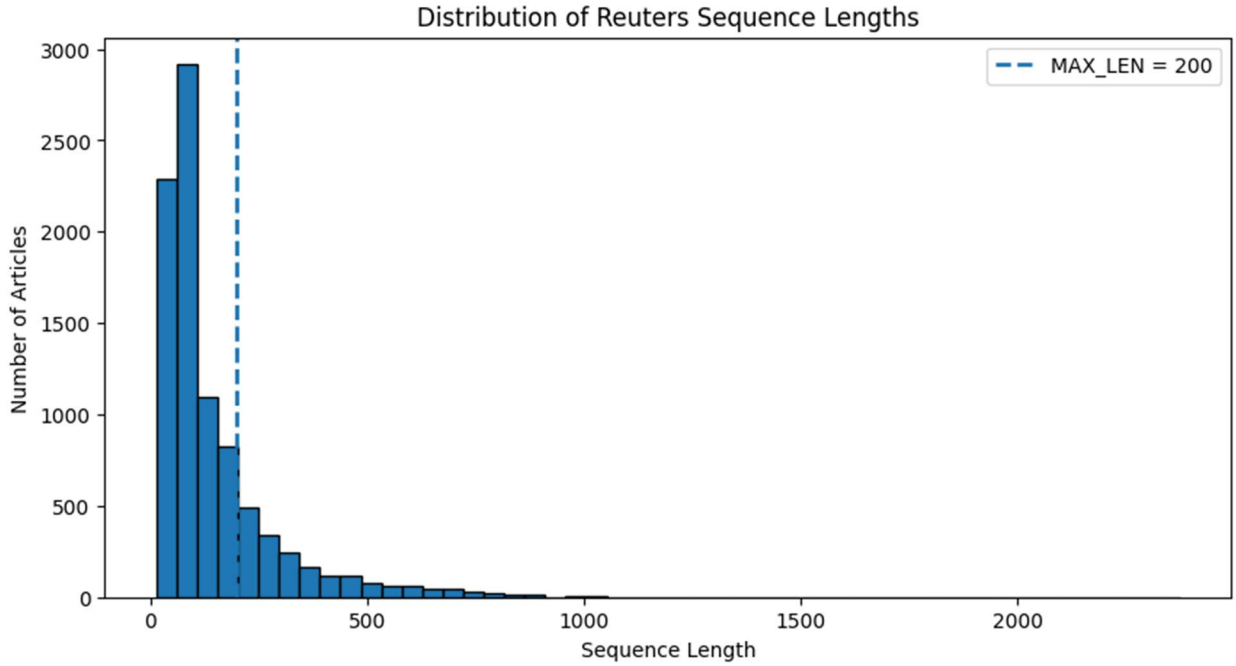
DATASET INFORMATION
Training samples : 8982
Testing samples : 2246
Number of classes: 46

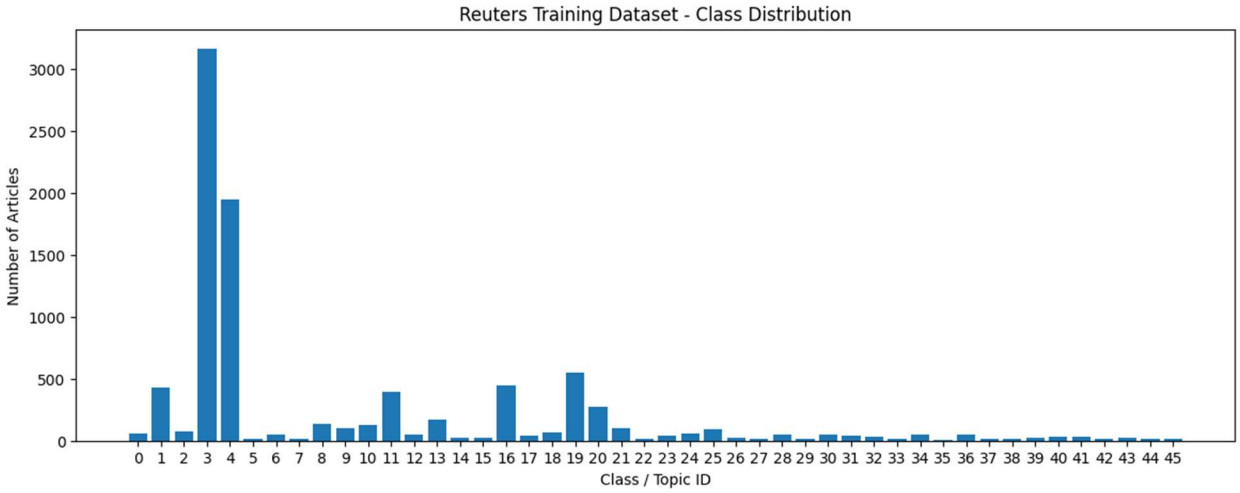
First training sample:
[1, 2, 2, 8, 43, 10, 447, 5, 25, 207, 270, 5, 3095, 111, 16, 369, 186, 90, 67, 7]

First training label:
3

SEQUENCE INFORMATION
Minimum sequence length : 13
Maximum sequence length : 2376
Average sequence length : 145.54

Median sequence length : 95.0





Padding sequences...
Training shape: (8982, 200)
Testing shape : (2246, 200)

TRAINING: RNN

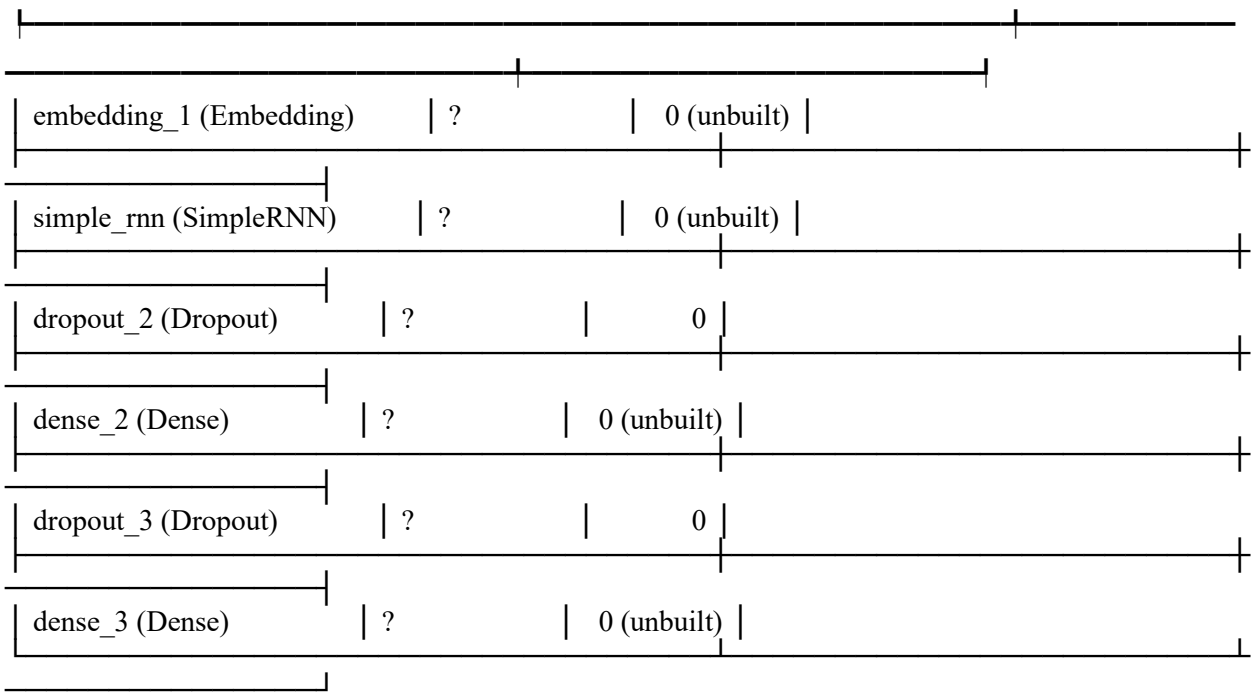
Model Summary:

/usr/local/lib/python3.13/dist-packages/keras/src/layers/core/embedding.py:100: UserWarning: Argument `input_length` is deprecated. Just remove it.

warnings.warn(

Model: "sequential_1"

Layer (type)	Output Shape	Param #



Total params: 0 (0.00 B)
Trainable params: 0 (0.00 B)
Non-trainable params: 0 (0.00 B)
Epoch 1/5
57/57 ━━━━━━━━━━━━━━━━━━━━ 24s 388ms/step - accuracy: 0.3076 - loss: 2.7554 - val_accuracy: 0.2220 - val_loss: 2.4720
Epoch 2/5
57/57 ━━━━━━━━━━━━━━━━━━━━ 20s 356ms/step - accuracy: 0.3382 - loss: 2.4755 - val_accuracy: 0.4090 - val_loss: 2.3159
Epoch 3/5
57/57 ━━━━━━━━━━━━━━━━━━━━ 22s 382ms/step - accuracy: 0.4058 - loss: 2.2274 - val_accuracy: 0.4430 - val_loss: 2.1307
Epoch 4/5
57/57 ━━━━━━━━━━━━━━━━━━━━ 22s 378ms/step - accuracy: 0.5100 - loss: 1.8709 - val_accuracy: 0.4469 - val_loss: 2.0422
Epoch 5/5
57/57 ━━━━━━━━━━━━━━━━━━━━ 21s 366ms/step - accuracy: 0.5951 - loss: 1.5459 - val_accuracy: 0.4591 - val_loss: 2.0511

RNN RESULTS
Accuracy : 0.4452
Precision: 0.4126
Recall : 0.4452
F1 Score : 0.3993
Training Time: 128.94 seconds
Parameters: 1324142

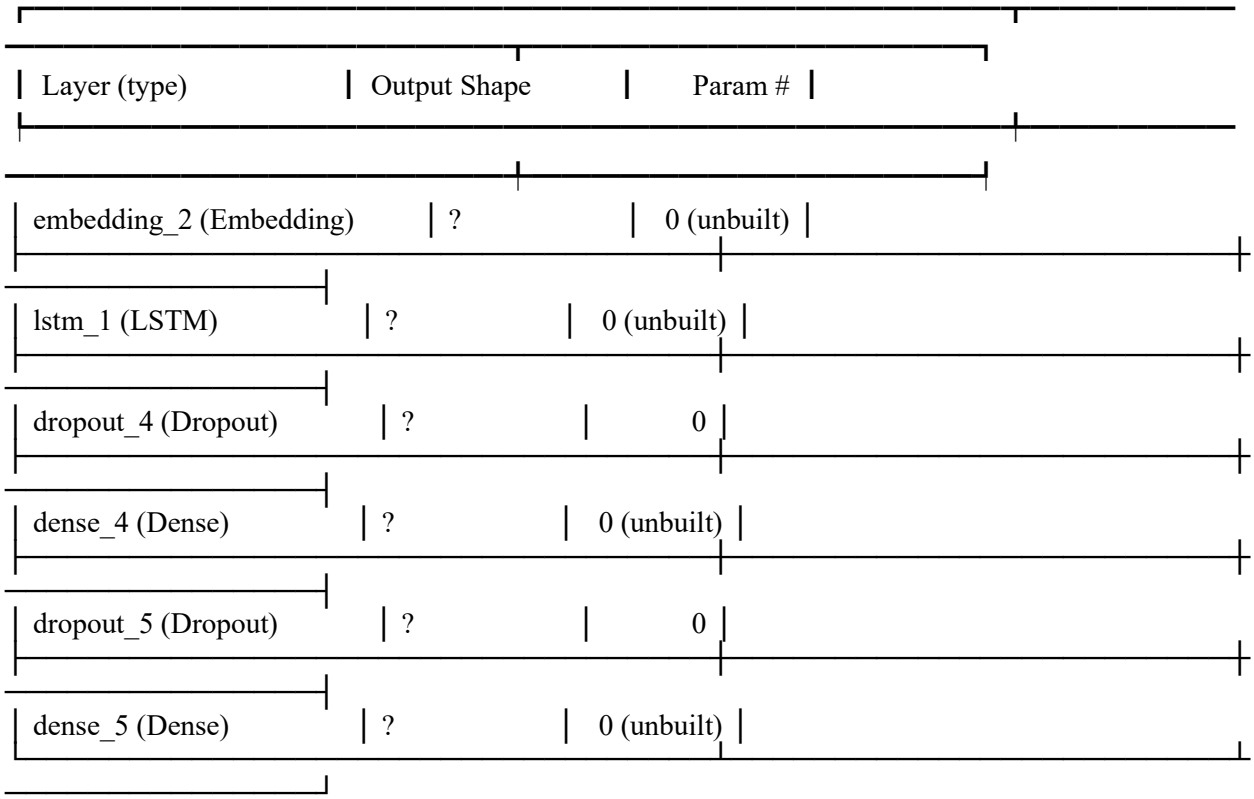
Classification Report:

	precision	recall	f1-score	support

0	0.00	0.00	0.00	12
1	0.50	0.03	0.05	105
2	0.00	0.00	0.00	20
3	0.79	0.69	0.74	813
4	0.41	0.67	0.51	474
5	0.00	0.00	0.00	5
6	0.00	0.00	0.00	14
7	0.00	0.00	0.00	3
8	0.00	0.00	0.00	38
9	0.00	0.00	0.00	25
10	0.00	0.00	0.00	30
11	0.00	0.00	0.00	83
12	0.00	0.00	0.00	13
13	0.00	0.00	0.00	37
14	0.00	0.00	0.00	2
15	0.00	0.00	0.00	9
16	0.13	0.15	0.14	99
17	0.00	0.00	0.00	12
18	0.00	0.00	0.00	20
19	0.17	0.79	0.28	133
20	0.00	0.00	0.00	70
21	0.00	0.00	0.00	27
22	0.00	0.00	0.00	7
23	0.00	0.00	0.00	12
24	0.00	0.00	0.00	19
25	0.00	0.00	0.00	31
26	0.00	0.00	0.00	8
27	0.00	0.00	0.00	4
28	0.00	0.00	0.00	10
29	0.00	0.00	0.00	4
30	0.00	0.00	0.00	12
31	0.00	0.00	0.00	13
32	0.00	0.00	0.00	10
33	0.00	0.00	0.00	5
34	0.00	0.00	0.00	7
35	0.00	0.00	0.00	6
36	0.00	0.00	0.00	11
37	0.00	0.00	0.00	2
38	0.00	0.00	0.00	3
39	0.00	0.00	0.00	5
40	0.00	0.00	0.00	10
41	0.00	0.00	0.00	8
42	0.00	0.00	0.00	3
43	0.00	0.00	0.00	6
44	0.00	0.00	0.00	5
45	0.00	0.00	0.00	1
accuracy			0.45	2246
macro avg	0.04	0.05	0.04	2246
weighted avg	0.41	0.45	0.40	2246

TRAINING: LSTM

Model Summary:
/usr/local/lib/python3.13/dist-packages/keras/src/layers/core/embedding.py:100: UserWarning: Argument `input_length` is deprecated. Just remove it.
  warnings.warn(
Model: "sequential_2"



Layer (type)	Output Shape	Param #
embedding_2 (Embedding)	?	0 (unbuilt)
lstm_1 (LSTM)	?	0 (unbuilt)
dropout_4 (Dropout)	?	0
dense_4 (Dense)	?	0 (unbuilt)
dropout_5 (Dropout)	?	0
dense_5 (Dense)	?	0 (unbuilt)

 Total params: 0 (0.00 B)
 Trainable params: 0 (0.00 B)
 Non-trainable params: 0 (0.00 B)
Epoch 1/5
57/57 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 53s 885ms/step - accuracy: 0.3620 - loss: 2.6895 - val_accuracy: 0.4385 - val_loss: 2.1387
Epoch 2/5
57/57 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 50s 878ms/step - accuracy: 0.4582 - loss: 2.0193 - val_accuracy: 0.4752 - val_loss: 1.8542
Epoch 3/5
57/57 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 82s 873ms/step - accuracy: 0.4799 - loss: 1.8646 - val_accuracy: 0.5320 - val_loss: 1.7380
Epoch 4/5
57/57 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 49s 868ms/step - accuracy: 0.5338 - loss: 1.7355 - val_accuracy: 0.5381 - val_loss: 1.7145
Epoch 5/5

57/57 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 82s 875ms/step - accuracy: 0.5557 - loss: 1.6208 - val_accuracy: 0.5799 - val_loss: 1.6360

LSTM RESULTS
Accuracy : 0.5726
Precision: 0.5178
Recall : 0.5726
F1 Score : 0.5323
Training Time: 348.47 seconds
Parameters: 1422830

Classification Report:

	precision	recall	f1-score	support
0	0.00	0.00	0.00	12
1	0.12	0.12	0.12	105
2	0.00	0.00	0.00	20
3	0.91	0.93	0.92	813
4	0.77	0.86	0.81	474
5	0.00	0.00	0.00	5
6	1.00	0.14	0.25	14
7	0.00	0.00	0.00	3
8	0.00	0.00	0.00	38
9	0.00	0.00	0.00	25
10	0.00	0.00	0.00	30
11	0.07	0.12	0.09	83
12	0.00	0.00	0.00	13
13	0.00	0.00	0.00	37
14	0.00	0.00	0.00	2
15	0.00	0.00	0.00	9
16	0.00	0.00	0.00	99
17	0.00	0.00	0.00	12
18	0.00	0.00	0.00	20
19	0.17	0.70	0.27	133
20	0.00	0.00	0.00	70
21	0.00	0.00	0.00	27
22	0.00	0.00	0.00	7
23	0.00	0.00	0.00	12
24	0.00	0.00	0.00	19
25	0.08	0.23	0.12	31
26	0.00	0.00	0.00	8
27	0.00	0.00	0.00	4
28	0.00	0.00	0.00	10
29	0.00	0.00	0.00	4
30	0.00	0.00	0.00	12
31	0.00	0.00	0.00	13
32	0.00	0.00	0.00	10
33	0.00	0.00	0.00	5
34	0.00	0.00	0.00	7
35	0.00	0.00	0.00	6
36	0.00	0.00	0.00	11

37	0.00	0.00	0.00	2
38	0.00	0.00	0.00	3
39	0.00	0.00	0.00	5
40	0.00	0.00	0.00	10
41	0.00	0.00	0.00	8
42	0.00	0.00	0.00	3
43	0.00	0.00	0.00	6
44	0.00	0.00	0.00	5
45	0.00	0.00	0.00	1
accuracy			0.57	2246
macro avg	0.07	0.07	0.06	2246
weighted avg	0.52	0.57	0.53	2246

TRAINING: GRU

Model Summary:
/usr/local/lib/python3.13/dist-packages/keras/src/layers/core/embedding.py:100: UserWarning: Argument `input_length` is deprecated. Just remove it.
  warnings.warn(
Model: "sequential_3"

Layer (type)	Output Shape	Param #
embedding_3 (Embedding)	?	0 (unbuilt)
gru (GRU)	?	0 (unbuilt)
dropout_6 (Dropout)	?	0
dense_6 (Dense)	?	0 (unbuilt)
dropout_7 (Dropout)	?	0
dense_7 (Dense)	?	0 (unbuilt)

Total params: 0 (0.00 B)
Trainable params: 0 (0.00 B)
Non-trainable params: 0 (0.00 B)

Epoch 1/5
57/57 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 52s 844ms/step - accuracy: 0.3447 - loss: 2.7332
- val_accuracy: 0.4669 - val_loss: 2.1056
Epoch 2/5
57/57 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 48s 848ms/step - accuracy: 0.4690 - loss: 2.0252
- val_accuracy: 0.4808 - val_loss: 1.9636
Epoch 3/5
57/57 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 49s 862ms/step - accuracy: 0.5072 - loss: 1.8270
- val_accuracy: 0.5264 - val_loss: 1.8112
Epoch 4/5
57/57 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 47s 814ms/step - accuracy: 0.5450 - loss: 1.6742
- val_accuracy: 0.5231 - val_loss: 1.8339
Epoch 5/5
57/57 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 48s 839ms/step - accuracy: 0.5843 - loss: 1.5760
- val_accuracy: 0.5726 - val_loss: 1.7151

GRU RESULTS
Accuracy : 0.5677
Precision: 0.5215
Recall : 0.5677
F1 Score : 0.528
Training Time: 243.84 seconds
Parameters: 1390318

Classification Report:

	precision	recall	f1-score	support
0	0.00	0.00	0.00	12
1	0.18	0.68	0.28	105
2	0.00	0.00	0.00	20
3	0.94	0.93	0.93	813
4	0.77	0.76	0.76	474
5	0.00	0.00	0.00	5
6	0.00	0.00	0.00	14
7	0.00	0.00	0.00	3
8	0.00	0.00	0.00	38
9	0.00	0.00	0.00	25
10	0.00	0.00	0.00	30
11	0.00	0.00	0.00	83
12	0.00	0.00	0.00	13
13	0.00	0.00	0.00	37
14	0.00	0.00	0.00	2
15	0.00	0.00	0.00	9
16	0.00	0.00	0.00	99
17	0.00	0.00	0.00	12
18	0.00	0.00	0.00	20
19	0.16	0.69	0.26	133
20	0.00	0.00	0.00	70
21	0.00	0.00	0.00	27
22	0.00	0.00	0.00	7

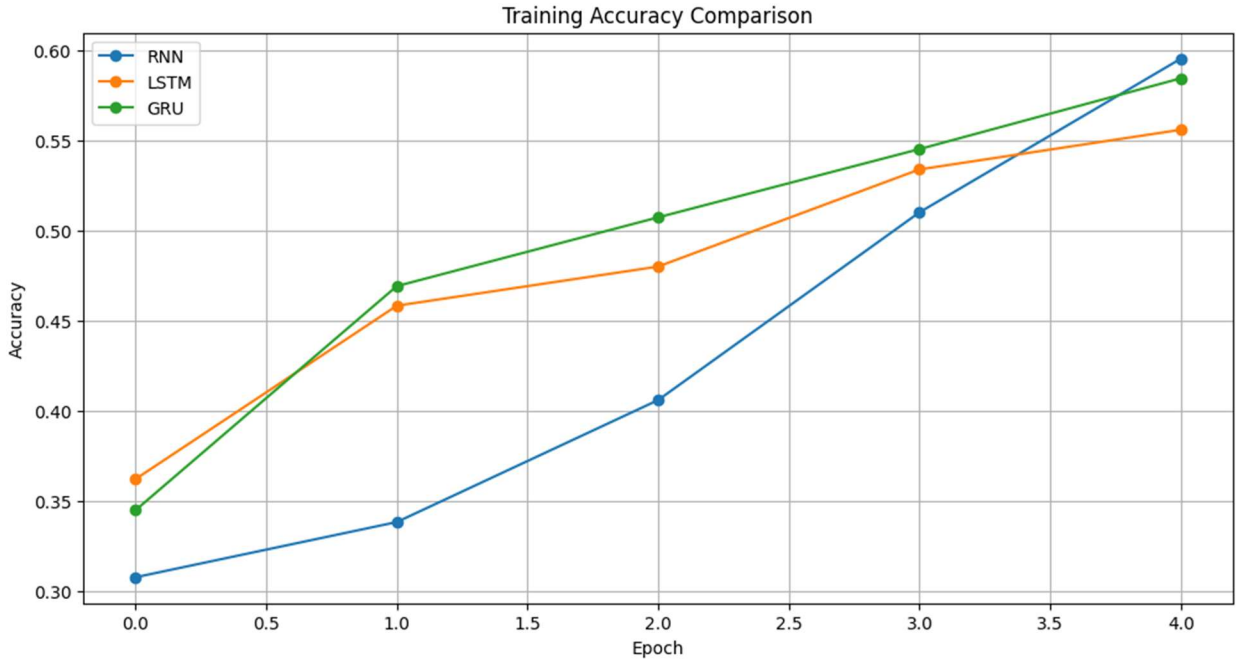
23	0.00	0.00	0.00	12
24	0.00	0.00	0.00	19
25	0.00	0.00	0.00	31
26	0.00	0.00	0.00	8
27	0.00	0.00	0.00	4
28	0.00	0.00	0.00	10
29	0.00	0.00	0.00	4
30	0.00	0.00	0.00	12
31	0.00	0.00	0.00	13
32	0.00	0.00	0.00	10
33	0.00	0.00	0.00	5
34	0.00	0.00	0.00	7
35	0.00	0.00	0.00	6
36	0.00	0.00	0.00	11
37	0.00	0.00	0.00	2
38	0.00	0.00	0.00	3
39	0.00	0.00	0.00	5
40	0.00	0.00	0.00	10
41	0.00	0.00	0.00	8
42	0.00	0.00	0.00	3
43	0.00	0.00	0.00	6
44	0.00	0.00	0.00	5
45	0.00	0.00	0.00	1
accuracy			0.57	2246
macro avg	0.04	0.07	0.05	2246
weighted avg	0.52	0.57	0.53	2246

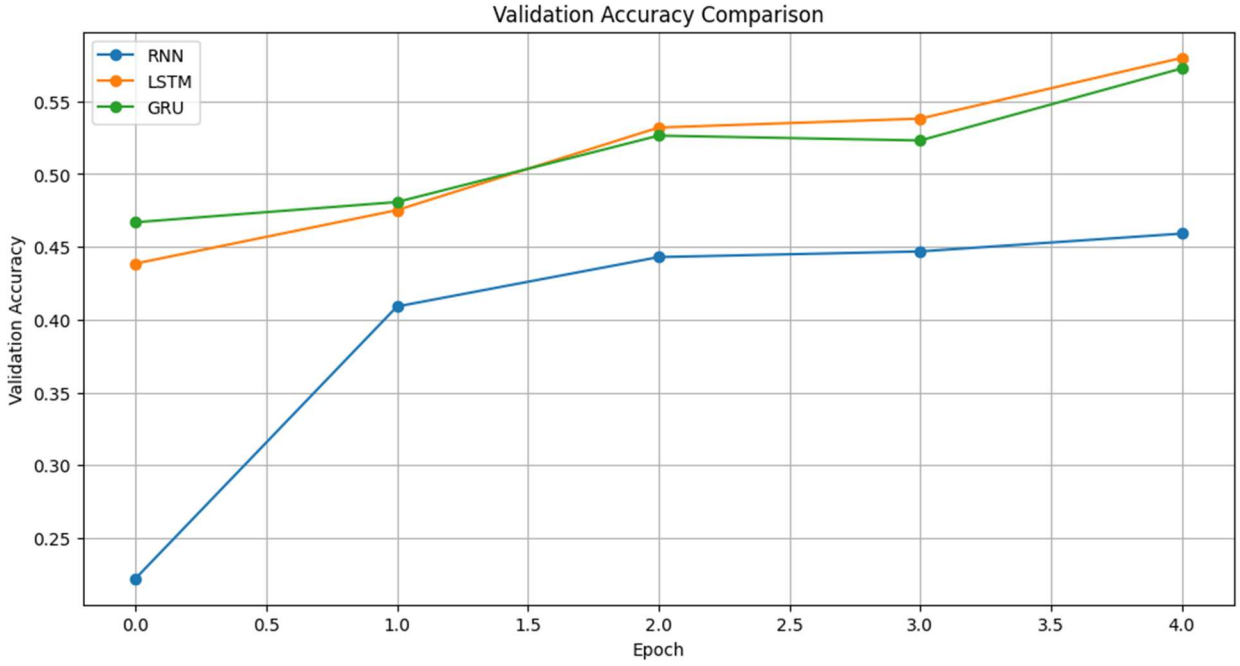
FINAL MODEL COMPARISON

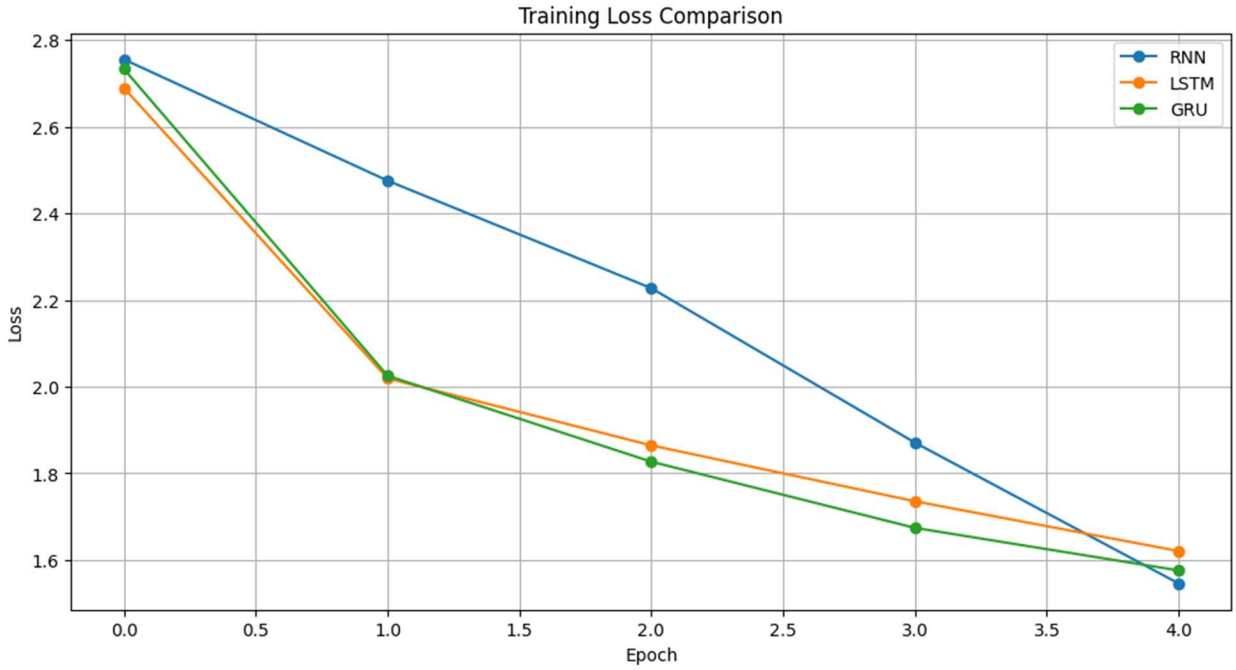
	Model	Accuracy	Precision	Recall	F1 Score	Training Time (sec)	Parameters
0	RNN	0.445236	0.412647	0.445236	0.399313	128.938954	1324142
1	LSTM	0.572573	0.517804	0.572573	0.532325	348.466768	1422830
2	GRU	0.567676	0.521543	0.567676	0.527975	243.844408	1390318

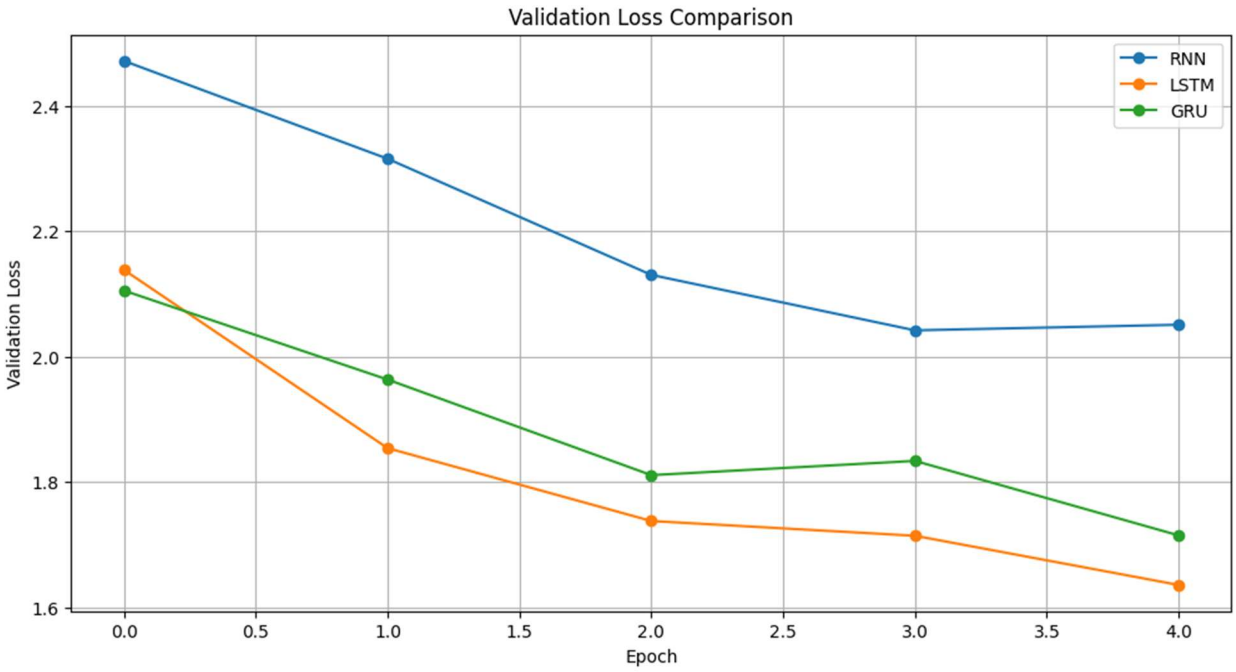
Formatted Results:

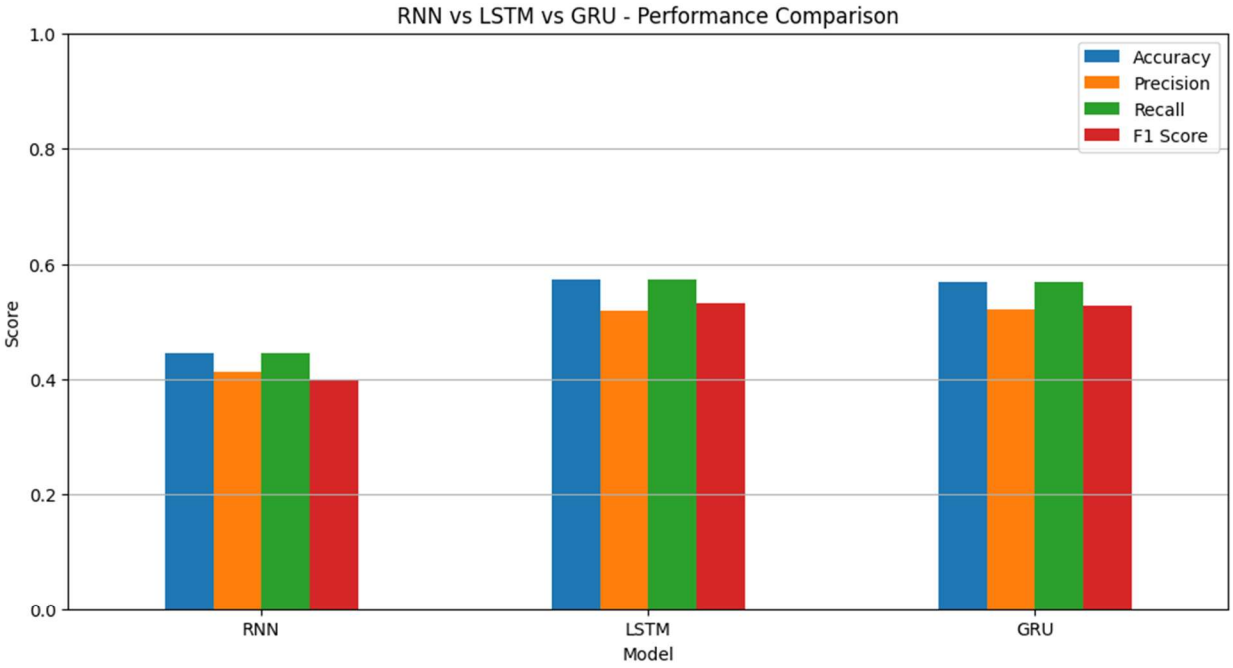
	Model	Accuracy	Precision	Recall	F1 Score	Training Time (sec)	Parameters
0	RNN	0.4452	0.4126	0.4452	0.3993	128.94	1324142
1	LSTM	0.5726	0.5178	0.5726	0.5323	348.47	1422830
2	GRU	0.5677	0.5215	0.5677	0.5280	243.84	1390318

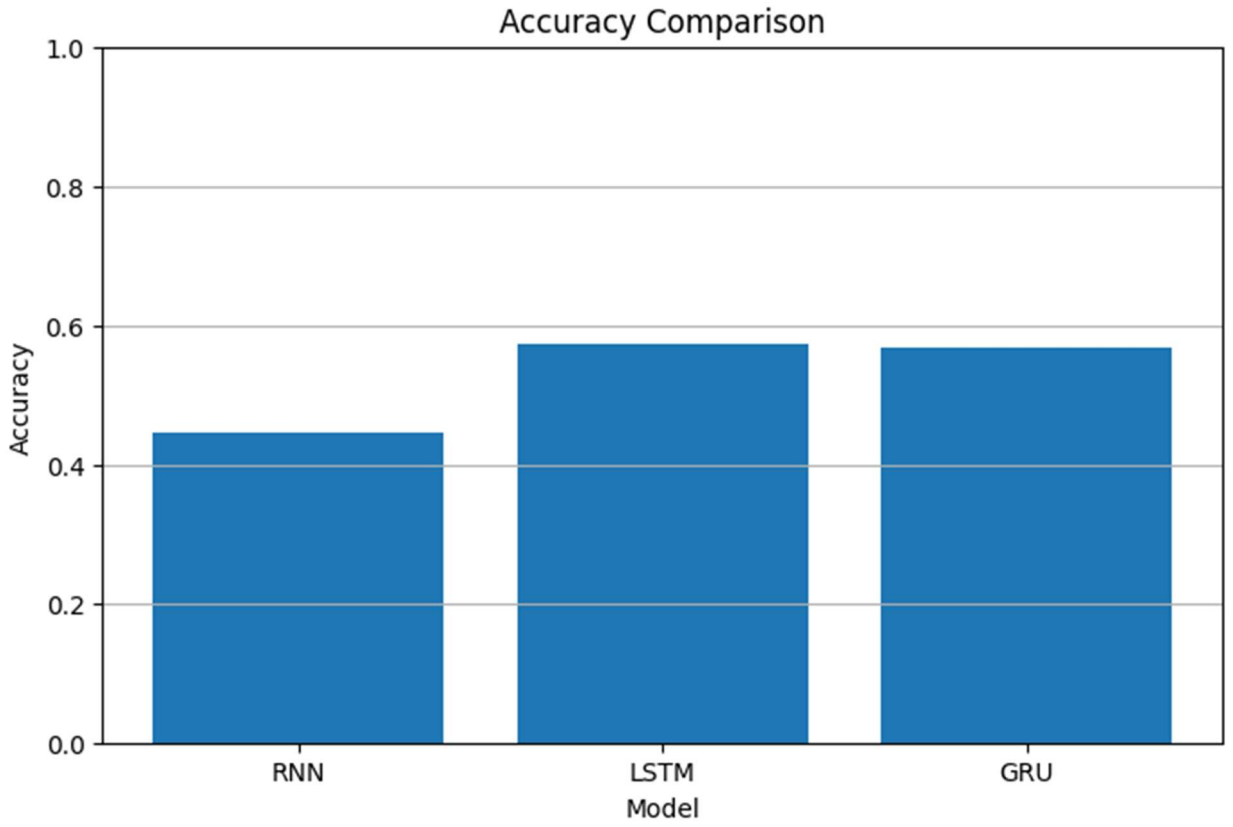

Training Accuracy Comparison
RNN
LSTM
GRU
Accuracy
Epoch
0.60
0.55
0.50
0.45
0.40
0.35
0.30
0.0
0.5
1.0
1.5
2.0
2.5
3.0
3.5
4.0

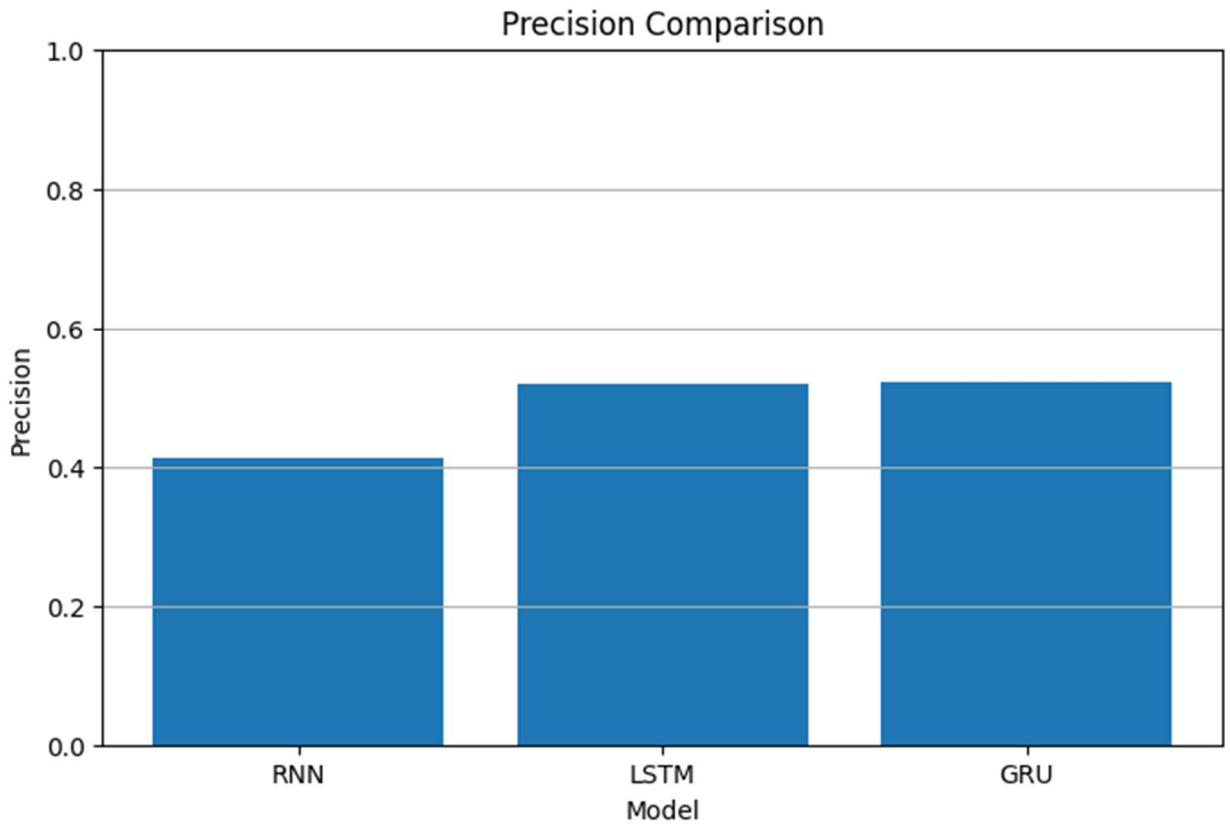

Validation Accuracy Comparison
RNN
LSTM
GRU
Validation Accuracy
Epoch
0.55
0.50
0.45
0.40
0.35
0.30
0.25
0.0
0.5
1.0
1.5
2.0
2.5
3.0
3.5
4.0

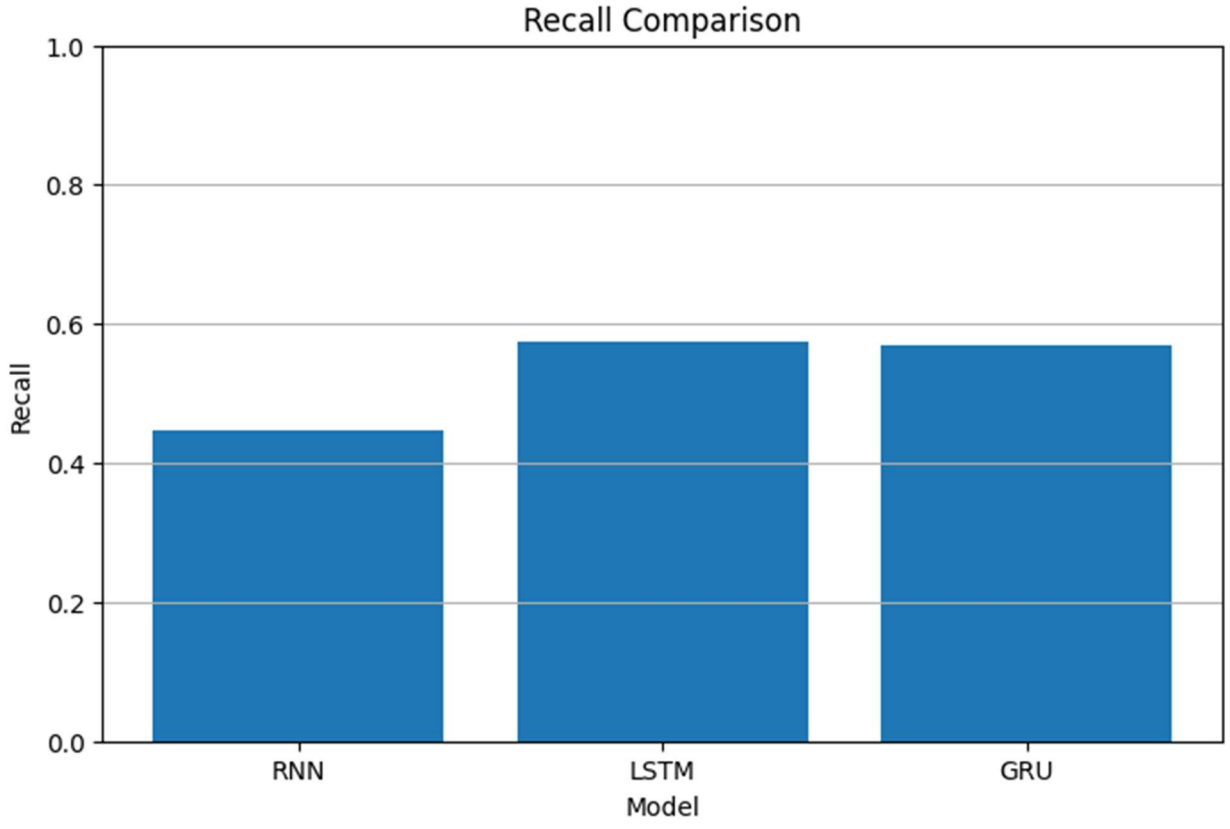

Training Loss Comparison
RNN
LSTM
GRU
Loss
Epoch
2.8
2.6
2.4
2.2
2.0
1.8
1.6
0.0
0.5
1.0
1.5
2.0
2.5
3.0
3.5
4.0

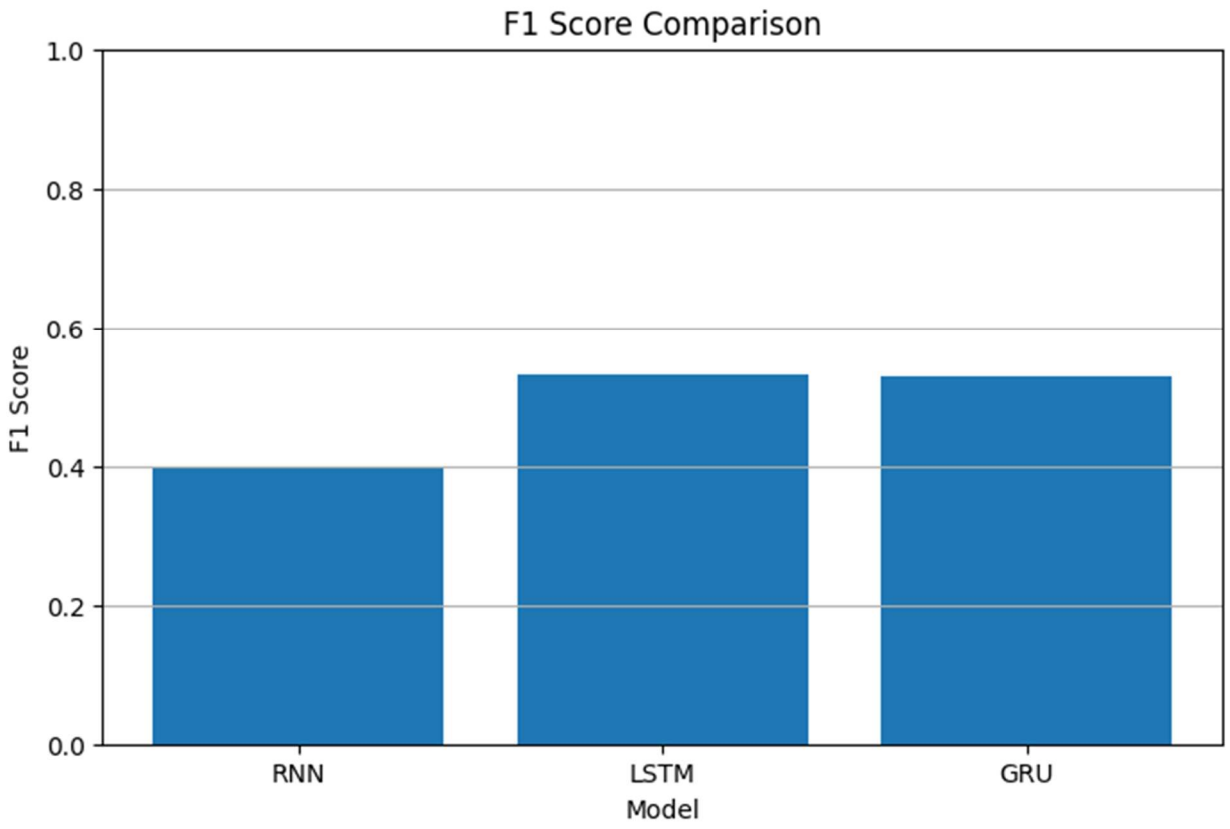

Validation Loss Comparison
RNN
LSTM
GRU
Validation Loss
Epoch
2.4
2.2
2.0
1.8
1.6
0.0
0.5
1.0
1.5
2.0
2.5
3.0
3.5
4.0

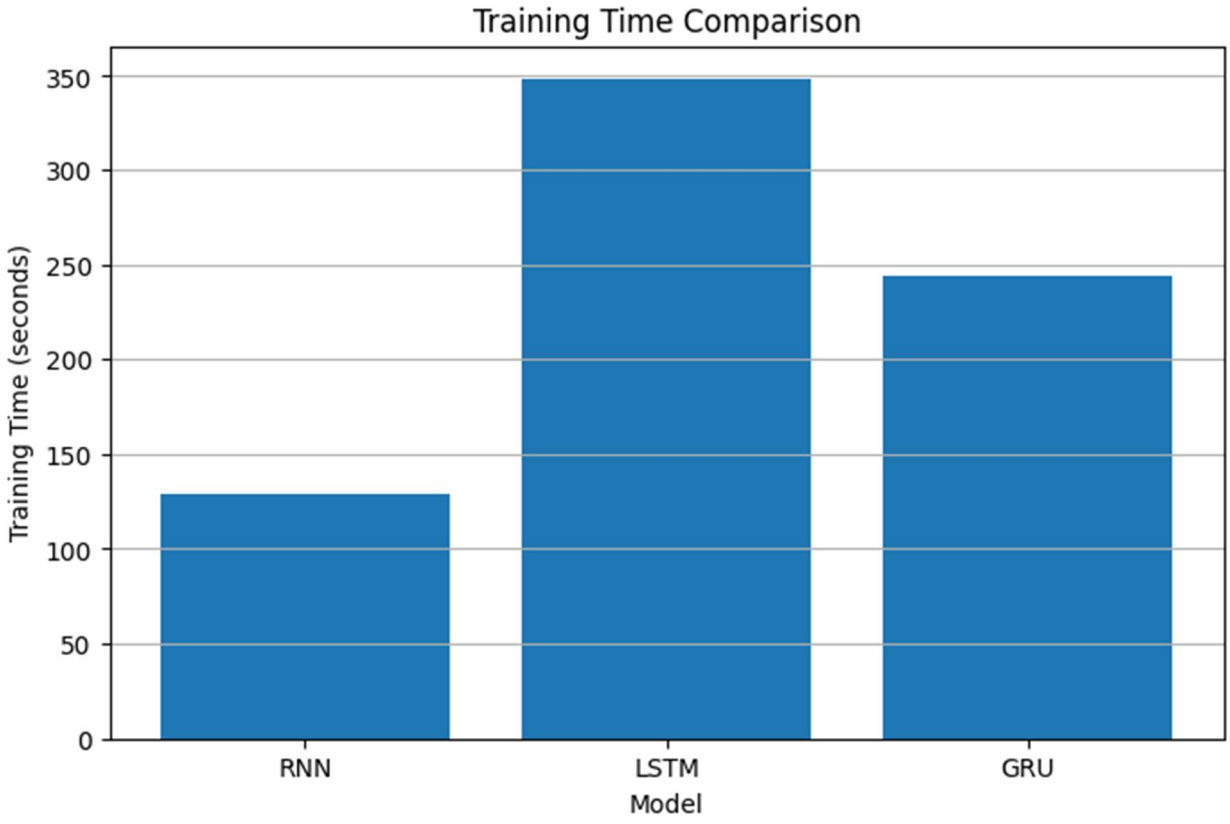

RNN vs LSTM vs GRU - Performance Comparison
Accuracy
Precision
Recall
F1 Score
Score
1.0
0.8
0.6
0.4
0.2
0.0
RNN
LSTM
GRU
Model

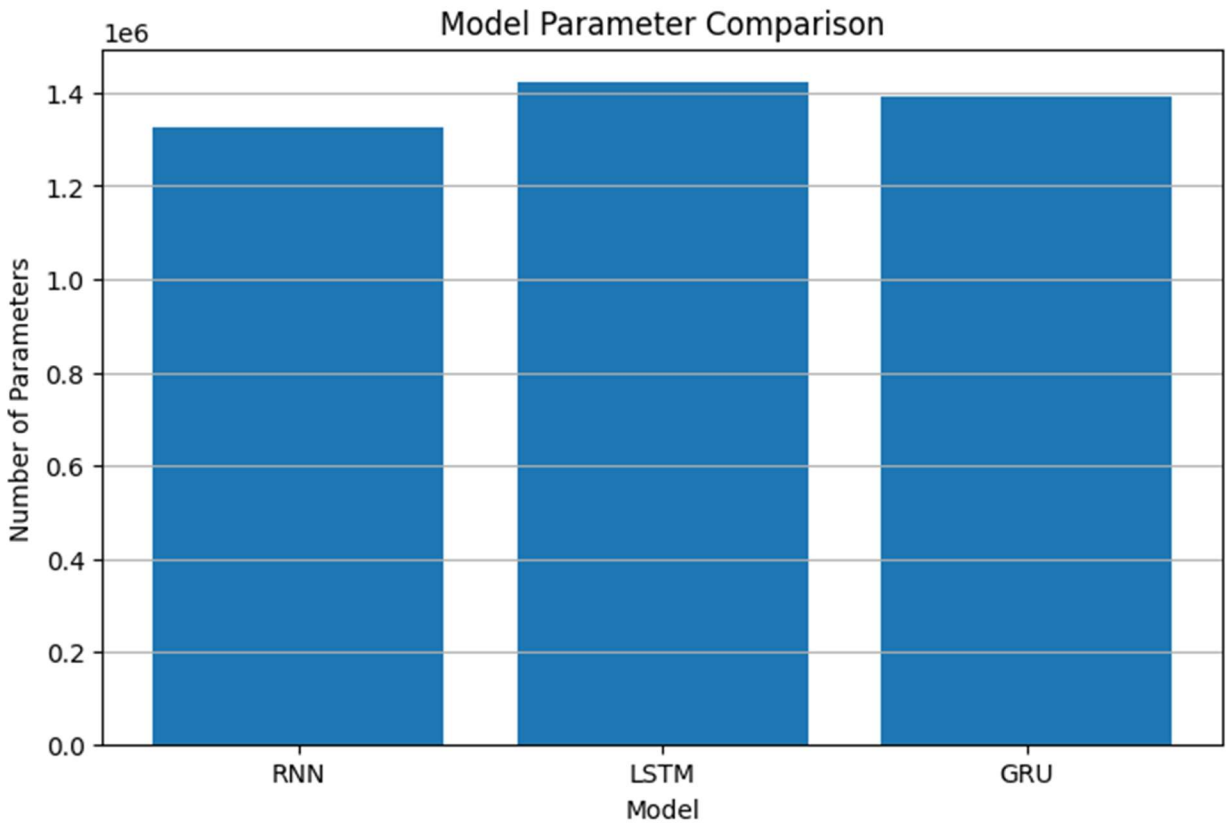

Accuracy Comparison
Accuracy
1.0
0.8
0.6
0.4
0.2
0.0
RNN
LSTM
GRU
Model

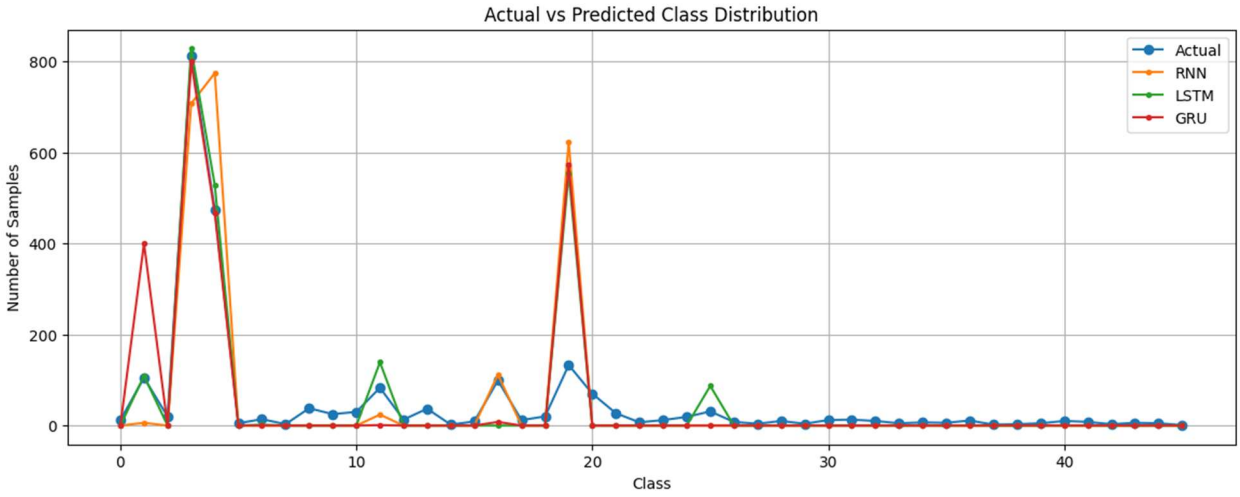

Precision Comparison
1.0
0.8
0.6
0.4
0.2
0.0
Precision
RNN
LSTM
GRU
Model


Recall Comparison
1.0
0.8
0.6
0.4
0.2
0.0
Recall
RNN
LSTM
GRU
Model


F1 Score Comparison
F1 Score
1.0
0.8
0.6
0.4
0.2
0.0
RNN
LSTM
GRU
Model


Training Time Comparison
Training Time (seconds)
350
300
250
200
150
100
50
0
RNN
LSTM
GRU
Model





BEST PERFORMANCE BY METRIC
Accuracy: LSTM (0.5726)
Precision: GRU (0.5215)
Recall: LSTM (0.5726)
F1 Score: LSTM (0.5323)